# Software Engineering

BSc Computer Science - Durham University

Matthew Emerson

November 2026

# Contents

**1 Module Introduction** **1**
1.1 Assessment . . . 1
1.2 Timetable . . . 1

**2 Project Management** **1**
2.1 Your Team, Formation, and Leadership . . . 1
2.2 Team Engagement, Negotiation, and Ideation . . . 2
2.3 Planning, Responsibilities, Milestones, and Risks . . . 2

**3 Software Engineering** **3**
3.1 The Software Development Lifecycle . . . 3
3.2 Agile Development in Practice . . . 4
3.3 Project Requirements . . . 4

**4 Legal, Social, Ethical, and Professional Issues** **6**
4.1 Introduction to Software Engineering and Software Engineering Ethics . . . 7

**5 Group Project** **7**

**I Appendix** **8**

**6 Bibliography** **8**
6.1 Module Professors . . . 8
6.2 References . . . 9
6.3 List of Figures . . . 9
6.4 Reading List . . . 9

# 1 Module Introduction

Software Engineering has the module code COMP2281 and is led by Dr Craig Stewart.

## 1.1 Assessment

This module is assessed through a group project, the deadlines for this can be seen in Table 1. Additional formatives are also shown in italics.

| Title | Opening | Due | Feedback | Professor | Weight |
|---|---|---|---|---|---|
| Peer Evaluation 1 | 16th Oct | 19th Nov | 10th Dec | Dr Craig Stewart | 1.67% |
| Requirement Documentation | 16th Oct | 19th Nov | 17th Dec | Dr Craig Stewart | 15% |
| Peer Evaluation 2 | 16th Oct | 11th Feb | 11th Mar | Dr Craig Stewart | 1.67% |
| Product Presentation | Week 16 | - | 18th Mar | Dr Craig Stewart | 15% |
| Technical Report | 5th Feb | 18th Mar | 6th May | Dr Craig Stewart | 30% |
| Peer Evaluation 3 | 5th Feb | 18th Mar | 6th May | Dr Craig Stewart | 1.67% |
| Git Commit | 5th Feb | 29th Apr | 1st Jul | Dr Craig Stewart | 5% |
| Project Handover | 5th Feb | 29th Apr | 1st Jul | Dr Craig Stewart | 10% |
| Reflective Experience Report | 5th Feb | 6th May | 1st Jul | Dr Craig Stewart | 20% |

Table 1: Assessments for Software Engineering.

## 1.2 Timetable

This module is taught through two one-hour lectures and one two-hour computer class each week as seen below in Table 2.

| Day | Time | Class Type | Location | Lecturer |
|---|---|---|---|---|
| Tuesday | 09:00 - 10:00 | Lecture | TLC033 | Dr Nelly Bencomo |
| Wednesday | 11:00 - 12:00 | Lecture | TLC033 | Dr Nelly Bencomo |
| Thursday | 11:00 - 13:00 | Practical | CC-0007 | Dr Gagangeet Aujla |

Table 2: Michaelmas Term Timetable for Software Engineering.

These lectures are taught jointly by Dr Craig Stewart and Dr Nelly Bencomo.

# 2 Project Management

## 2.1 Your Team, Formation, and Leadership

This lecture was taught by Dr Nelly Bencomo.

Belbin (https://www.belbin.com/about/belbin-team-roles) defines nine roles someone can take in a team. These are:

- The Innovator who solves problems as the "ideas guy" of the group.
- The Resource Investigator who contacts external people/companies to bring additional resources to the group.
- The Coordinator who guides and controls the team. They know their team's abilities well.
- The Shaper who is a demanding and confrontational leader who pushes their members to excel.
- The Monitor/Evaluator who analyses proposed solutions and choices.
- The Team Worker who facilitates team functions and mediates issues.


*LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson*

- The Implementor who implements the agreed upon plans.
- The Completer/Finisher who focuses on details and meeting deadlines.

After assigning roles and completing skill evaluation tests, the team can then create a CPD plan which identifies existing skills, areas for development, and a plan for doing that.

Maslow, A. (2016) splits competency into four categories: unconscious incompetence, conscious incompetence, unconscious competence, and finally conscious competence.

## 2.2 Team Engagement, Negotiation, and Ideation

This lecture was taught by Dr Nelly Bencomo.

When negotiating, you are more likely to be successful if you have the following factors: legitimacy, confidence, courtesy, adaptation to the other party, rapport, incentives/trade-offs, and research into the bigger picture.

## 2.3 Planning, Responsibilities, Milestones, and Risks

This lecture was taught by Dr Nelly Bencomo.

A simple task becomes a project when it produces a unique outcome, a finite timeline, interdependant work, and some level of uncertainty.

The project triangle states that there are always trade-offs between scope, resources, and time. This is because quality is never optional.

### 2.3.1 Planning

Plan outcomes before tasks:

- Goal: What useful result will the client recieve?
- Deliverable: What evidence or artefact demonstrates it?
- Milestone: When will we check progress?

Never adjust deadlines - its easier to instead negotiate scope.

RACI: A responsibilty assignment matrix for making decisions and producing work.

- Responsible: The person that does the work.
- Accountable: The person that owns the outcome/approval.
- Consulted: The person that provides input before action.
- Informed: The person who recieves progress updates.

| Activity | Lead | Developer | Tester | Client |
|---|---|---|---|---|
| Agree Prject Scope | R | C | C | A |
| Implement a Feature | A | R | C | I |
| Test a Feature | A | C | R | I |
| Demonstrate Prototype | A | R | R | C |

Table 3: RACI assignment matrix


*LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson*

Project Management Tools: Trello (time management), GitHub Projects, Jira (scrum, sprints, backlogs), and/or Microsoft Planner (team tasks and deadlines).

Assigning priority level, using impact (1-5: negligible, minor, moderate, major, crucial) and risk (1-5: negligible, low, moderate, high, massive). Then set priority bands (e.g., 1-5 is low, 6-14 is medium, 15-25 is high).

| | | | | | |
|---|---|---|---|---|---|
| **5** | 5 | 10 | 15 | 20 | 25 |
| **4** | 4 | 8 | 12 | 16 | 20 |
| **3** | 3 | 6 | 9 | 12 | 15 |
| **2** | 2 | 4 | 6 | 8 | 10 |
| **1** | 1 | 2 | 3 | 4 | 5 |
| Risk ↑<br>Impact → | **1** | **2** | **3** | **4** | **5** |

Table 4: Assigning task priority in terms of risk and impact.

# 3 Software Engineering

## 3.1 The Software Development Lifecycle

This lecture was taught by Dr Nelly Bencomo.

### 3.1.1 Introduction to Software Development

**Definition 3.1.1.1**: The software development lifecycle is split into 6 stages: planning, analysis, design, implementation, testing and integration, and maintenance. This can be implemented in a number of ways:

- Waterfall: Planned, sequential stages. Good: Visible stage boundaries. Bad: Feedback can arrive late.
- V-Model: Connects development levels to verification and validation. Good: Explicit assurance. Bad: Changes can be costly.
- Incremental: Deliver useful functionality in successive parts. Good: Early value. Bad: Integration and architecture.
- Iterative: Refine a solution through repeated feedback. Good: Learning. Bad: Potential rework.
- Spiral: Cycles driven by risk assessment and reduction. Good: Tackles high risks early. Bad: Expertise and overhead.
- Agile: Adaptive planning and frequent valuable increments. Good: Responsiveness. Bad: Sustained collaboration required.

### 3.1.2 Agile

**Definition 3.1.2.1**: The Agile Manifesto (first used in 2001) defines their priorities as people (individuals and interactions over processes and tools), software (working software over comprehensive documentation), collaboration (customer collaboration over contract negotiation), and change (responding to change over following a plan).

It requires engineering discipline, implementing requirements (clarify and revisit user needs), design (make and review architecture decisions), quality (test and integrate continuously), and feedback (inspect results with stakeholders).

**Definition 3.1.2.2**: A scrum is a set period of time within an project and involves a product owner, scrum master, and development team.


*LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson*

1. When beginning a scrum, the product owner will usually provide a product backlog which is taken into a sprint planning meeting.
2. Here, the scrum master discusses with the development team and sets out the work to be completed during the upcoming sprint.
3. The team then start a sprint and work through as much of the backlog as possible.
4. At the end of a chosen period of time (usually 1-4 weeks), the team carry a sprint review, identifying work done and issues preventing progress, before potentially choosing to release a product increment.
5. Ideally there should also be regular checkups throughout the sprint to identify progress and potential issues.

## 3.2 Agile Development in Practice

This lecture was taught by Dr Nelly Bencomo.

### 3.2.1 Approaches to Programming

**Definition 3.2.1.1**: Extreme Programming (XP) is built from five core values: communication, simplicity, feedback, courage, and respect. Whilst doing this, we take engineering discipline into account by making small releases early to get client feedback, making continuous integrations to reduce surprises when merging teammates' work, and refactoring frequently to keep change affordable.

Generally, we split XP into 12 basic practices: the planning game, small releases, metaphor, simple design, test-first programming, refactoring, pair programming, continuous integration, collective ownership, on-site customer, 40-hour weeks, and an open workspace.

**Definition 3.2.1.2**: Pair programming is the practice of coding in pairs where one person is the driver and uses the keyboard, while the other is the observer who discusses with the driver and ensures the code makes sense. The pair swap roles on a regular basis. This provides learning benefits for less-experienced programmers.

**Definition 3.2.1.3**: Test-first development is the practice of specifying an observable test, implementing the smallest change that passes the test before running all other existing tests and then refactoring the solution until all other tests are green.

### 3.2.2 Task Management

**Definition 3.2.2.1**: Agile projects generally utilise a Kanban board to keep track of project progress. It consists of columns representing the workflow stages (to-do, in progress, review, done, etc.) and cards representing individual tasks. The "in progress" column is often limited to a set number of items to reduce the likelihood of burnout.

## 3.3 Project Requirements

### 3.3.1 Requirements I

This lecture was taught by Professor Pete Sawyer, taking content from his book: Requirement Engineering: A Good Practice Guide.

Requirements are important for a project as having wrong requirements can lead to the resulting software not being fit for purpose and errors can propogate and become costly to fix later. It is important to validate early to ensure you have the right requirements. As such, requirements statements become about communication. Stakeholders need confidence you understand what they want and developers need to know what to build.

**Definition 3.3.1.1**: Functional requirements give details about features the software provides.


*LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson*

**Definition 3.3.1.2**: Non-functional requirements give details about qualities or constraints for the project. They can be dynamic (reliability, accuracy, availability, etc.) or static (maintainability, compliance, etc.). They often act as design constraints such as budget or time.

**Definition 3.3.1.3**: People or organisations with requirements are stakeholders. Users perform the taret tasks, clients/customers pay for development, external/regulatory bodies ensure compliance, and other systems provide interaction requirements. As such, it is good practice to identify and document the software's stakeholders.

When trying to elicit requirements from stakeholders you can conduct interviews, observations, workshops, or focus groups. You can also add requirements after the software has deployed through bug reports and feature requests. UML models can also be useful to contextualise workflows and the organisation's architecture / stakeholders' requirements.

Verification ensures that requirements have been met. To do this, each requirement should have a test case. This can be challenging for NFRs as they can easily become poorly defined.

### 3.3.2 Requirements II

This lecture was taught by Dr Nelly Bencomo.

**Definition 3.3.2.1**: A software requirement is a functional or non-functional need to be implemented in the system.

- A functional requirement could provide a particular service to the user.
- A non-functional requirement can be a performance requirement.

**Definition 3.3.2.2**: Non-functional requirements are characteristics of the system which cannot be expressed as functions (e.g., maintainability, portability, security, etc.). These characteristics can be measurable (i.e., response time must be under 10 ms 95% of the time). Constraints are NFR (e.g., hardware to be used, OS, DBMS, etc.). Project management issues are also NFR (e.g., costs, schedules, logistics, etc.).

**Definition 3.3.2.3**: When gathering requirements, we can use the MoSCoW prioritisation method where requirements are sorted into one of "must haves", "should haves", "could haves", and "won't haves". We can then designate our workflow accordingly, giving maximum 60%, 20%, 20%, and 0% effort to these areas accordingly.

**Definition 3.3.2.4**: Behaviour driven development focuses on user stories and supports and encourages collaborative development. It is built on test-driven development and acceptance test driven development.

**Definition 3.3.2.5**: When presenting a requirement back to the user it should be human-readable by non-experts whilst still explaining the proposed workflow. Therefore, we use Gherkin which is the syntax language of Cucumber. This description is behavioural in nature but leads to a clearer understanding of what to code.

When outlining project requirements, we follow the following stages:

1. User needs domain information, existing system information, regulations, standards, etc.
2. Requirements Elicitation.
3. Requirements Analysis and Negotiation.
4. Requirements Documentation.
5. Requirements Validation (and optionally dropping back to any previous stage if needed).
6. This approach produces a requirement document and system specification which can then be approved by the client.


*LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson*

### 3.3.3 Requirements III

**Definition 3.3.3.1**: The role of requirements validation is to check that the right product is being built. It ensures that the software being developed (or changed) will satisfy the stakeholders and compares the software requirements specification against the stakeholders' goals and requirements. This is important as fixing a requirements error after delivery is much more expensive than fixing an implementation error.

To carry out this process, we use the following three steps throughout the process:

- *Elicitation*: Checking back with the elicitation sources (e.g., "So, are you saying that...?").
- *Analysis*: Checking that the domain description and requirements are correct.
- *Specification*: Checking that the defined system requirement will meet the user requirements under the assumptions of the domain/environment. Checking conformity to the well-formedness rules/standards.

**Definition 3.3.3.2**: The role of requirements verification is to check that the product is being built correctly. It ensures that each step followed in the process of building software yields the right products and checks the consistency of the software requirements specification artefacts and other software development products against the specification.

To carry out this process, we use the following steps throughout the process:

- *Validity Checks*: Does the system provide the functions which best support the customer's needs?
- *Consistency Checks*: Are there any requirements conflicts?
- *Completeness Checks*: Are all functions required by the customer included?
- *Realism Checks*: Can the requirements be implemented given available budget and technology?
- *Verifibility Checks*: Can the requirements be checked/tested?
- *Traceable*: Where/who did the requirement come from and rationale?

**Definition 3.3.3.3**: Each requirement carries a fit criteria. This is a benchmark or goal that the testing activity uses to determine whether the eventual solution satisfies the requirement. For functional requirements (FR), the function is either completed or not, and thus has no scales of measure. A non-functional requirement (NFR) is a quality the product must have, and fit criterion is a measure of that.

If such criteria cannot be easily expressed or quantified, then the requirement is likely to be ambiguous, incomplete, or incorrect. It is relatively easy to determine fit criteria for quality requirements that are naturally quantitative.

**Definition 3.3.3.4**: The prototyping process can start with those requirements which are poorly understood. This helps stakeholders understand what the system will do.

Prototypes for validation must be more complete than for elicitation as they must contain sufficient facilities for practical use to be made of it. This is because implementation of a number of features can help the user(s) to identify important features and those that are no so important.

We should translate requirements into a testable specification using the following process: understanding the client's need; creating a user story; setting fit criteria; writing Gherkin code to represent it; and then setting priority and rationale.

# 4 Legal, Social, Ethical, and Professional Issues

This recommended reading for this sub-topic is:

- The Software Engineering Code of Ethics and Professional Practice, ACM (2024).


*LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson*

## 4.1 Introduction to Software Engineering and Software Engineering Ethics

This lecture was taught by Dr Craig Stewart.

### 4.1.1 Introduction to Software Engineering

**Definition 4.1.1.1**: An ill-structured problem, as defined by (Simon, 1973), is: elusive, unbounded, not understandable, without end conditions. They require orientation and context, and have many stakeholders and many possible solutions.

**Definition 4.1.1.2**: A good problem solver needs the following:

- **Method**: Refers to a formal procedure for accomplishing a goal, typically independent of the tools used.
- **Tool**: An instrument or automated system for accomplishing something in a better way.
- **Procedure**: A combination of tools and techniques to produce a product.
- **Paradigm**: The philosophy or approach for building a project.

**Definition 4.1.1.3**: For this module, a stakeholder will generally fall into one of three roles:

- **Customer/Client**: The company, organisation, or person who pays for the software system.
- **Developer**: The company, organisation, or person who is building the software system.
- **User**: The person/people who will actually use the system.

### 4.1.2 Introduction to Ethics

**Definition 4.1.2.1**: The software engineering code of ethics is defined by Rost and Glass (2010) and summarised more thoroughly by the Association for Computing Machinery (2024).

# 5 Group Project


*LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson*

# Part I
# Appendix

## 6 Bibliography

### 6.1 Module Professors

**Module Lead**: Doctor Craig Stewart

- Focus: Adaptive systems, HCI, and user modelling.
- Email: craig.d.stewart@durham.ac.uk
- Office Hours: Friday 11:00 - 12:00.

**Lecturer**: Doctor Nelly Bencomo

- Focus: Software engineering for autonomous, self-adaptive, AI and ML systems.
- Email: nelly.bencomo@durham.ac.uk
- Office Hours: Email to make an appointment.

**Lecturer**: Doctor Gagangeet Aujla

- Focus: Data-driven and data-intensive cyber-physical systems and digital twins.
- Email: gagangeet.s.aujla@durham.ac.uk
- Office Hours: Email to make an appointment.

**Lecturer**: Doctor Gokberk Kabacaoglu

- Focus: Machine learning and numerical applications.
- Email: gokberk.kabacaoglu@durham.ac.uk
- Office Hours: Thursday 14:00 - 15:00.

**Lecturer**: Doctor Ryan Crosby

- Focus: The assessment of novice programmers.
- Email: ryan.crosby@durham.ac.uk
- Office Hours: Email to make an appointment.

**Lecturer**: Doctor Sukanya Pandey

- Focus: Graph theory.
- Email: sukanya.pandey@durham.ac.uk
- Office Hours: Email to make an appointment.

### 6.1.1 Guest Speakers

**Guest Speaker**: Professor Pete Sawyer

- Focus: Requirements engineering.
- Email: peter.h.sawyer@durham.ac.uk
- His Book: Requirements Engineering: A Good Practice Guide.

## 6.2 References

### 6.2.1 Legal, Social, Ethical, and Professional Issues

Simon, H.A. (1973). The Structure of Ill Structured Problems. Artificial Intelligence, 4(3-4).

Rost, J. and Glass, R.L. (2010). The Dark Side of Software Engineering: Evil on Computing Projects. Washington, DC: IEEE Computer Society.

Sommerville, I. and Sawyer, P. (1997). Requirements Engineering: A Good Practice Guide. John Wiley & Sons, Inc.

## 6.3 List of Figures

## 6.4 Reading List

### 6.4.1 Legal, Social, Ethical, and Professional Issues

Association for Computing Machinery (2024). The Software Engineering Code of Ethics and Professional Practice. [online] acm.org. Available at: https://www.acm.org/code-of-ethics/software-engineering-code [Accessed 7 Oct. 2026].